CNN Trained on CIFAR-10: Exploring Differences with Activations, Optimizations, Pools, and Resnet18 Architecture

Boedie Provencher

March 2026

Abstract

This project seeks to explore the strengths and weaknesses of different parameters on a custom-built convolutional neural network. We use different activations, optimizers, and pools to see which provided the greatest accuracy and understand how the different models engage image features. We also wanted to test how the model compares to the deep ResNet18 architecture, as it compares to a shallow CNN. The study highlights the individual capacity of each activation feature under the same optimization model to investigate differences. The results show ResNet outperforming the shallow CNN; with ReLu, Adam, and max pooling hyperparameters performing best within the CNN. Please note that the PyTorch documents were used to develop code and function organization, as well as in-class instruction.

1 Introduction

Convolutional neural networks are powerful machine learning tools for image generation, object discrimination, and overall data analysis and prediction. As unique activators change the assumptions of the core neural structure of the CNN, its important to see how they affect the model itself. This is further exemplified in the optimization and pooling of the model, which control weighted updating and spatial organization inside the CNN layers. Understanding how these differences affect the CNN and how they compare with the deeper and more complex ResNet architecture provides insight into how CNNs learn and model data.

This project develops a CNN with modifiable parameters and 4 learnable layers (a shallow network) with about 256 "neurons"; it is trained on the 10 image classes from the CIFAR-10 dataset in image classification. This dataset provides extensive coverage and allows the CNN to classify images on multiple object categories that provide meaningfully different visual features for a strong benchmark in image classification. We compare its performance along different

features with just the standard ResNet18 to recognize the differences between a shallow and deep architectural network.

ResNet uses residual learning properties that enable its neural networks to train over many layers with identity shortcut connections [1], this contrasts our shallow network as it goes through only 4 layers and lacks the refined neuronal connections that improve performance. We should expect to see ResNet outperform the shallow CNN on both optimizers (Adam, SGD) on the CIFAR-10 dataset as exemplified by [1].

2 Methodology

2.1 Experimental Configuration

We use the CIFAR-10 dataset for train-test data and comparison by focusing on tuning different hyperparameters; over 10 epochs per alteration including: architecture, activation type, optimizer type, and pooling. We analyze training loss curves, validation and training accuracy, overall test accuracy, confusion matrices for each experiment, and we observe the feature mapping differences between architectures. Please note we use a cross-entropy loss function for all training backpropagation.

2.2 Architectures

The study observes the overall differences in performance between a shallow and adaptive CNN configuration and the deeper learning ResNet model. We also take a peak 'under the hood', looking at what each model is focusing on in layer 1 of the convolution network of the 1st image in each class: focusing on the feature mapping and filter weights (see source code).

2.2.1 Shallow CNN

Our custom-configurable network trains with 2 conv feature extraction layers, and 2 connected classification layers for learning layers. Each convolution layer uses 3×3 kernel size and is followed by a nonlinear activation function and a pooling layer for spatial down-sampling. The resulting feature maps are flattened and passed through a fully connected layer with 256 units before producing a 10-class output. The architecture allows experimentation with different activation functions, optims, and pooling methods to evaluate their effect on classification performance.

2.2.2 ResNet18

A deep CNN with 18 learnable layers and residual connections. ResNet18 allows for deeper networks to train effectively by using skip connections, which help gradients propagate backward and reduce the vanishing gradient problem [1]. In our experiments, we adapted ResNet18 to CIFAR-10 by setting its classes to

10. This serves as a benchmark to compare our custom shallow CNN against a deeper architecture.

2.2.3 Feature/Filter Maps

Maps were obtained via hooking (Pytorch Docs) in Conv1 of the first image in each class after model training. Due to the size of the data the feature and filter maps are provided as a PDF (see source code).

2.3 Activations

We tested three different activation functions to introduce nonlinearity in the network, all trained using the Adam optimizer to ensure consistent evaluation.: ReLu (outputs zero for negative values and passes positive values unchanged), Sigmoid (outputs to 0-1), Tanh (outputs to -1 - 1).

2.4 Optimizers

We use Adam and SGD (Stochastic Gradient Descent). Note that we have RMSprop only in the case that an optimizer isn't defined at runtime for error handling (not used in this study).

2.5 Pooling

Pooling layers can reduce the spatial dimensions of feature maps in each model, ultimately helping to improve computational efficiency and reduce overfitting. We use maximum-"max"(maximum value) and average-"avg" (average value) pooling for this study.

2.6 DataSet

We train and test our CNN (and ResNet) on the CIFAR-10 as described in [1], with 50k training images, 10k test images over 10 unique classes. This provides strong diversity in image classification that strengthens train-test data and results.

3 Results

3.1 Training Loss

These graphs show the training loss curves for tuned hyperparameters and different architectures.

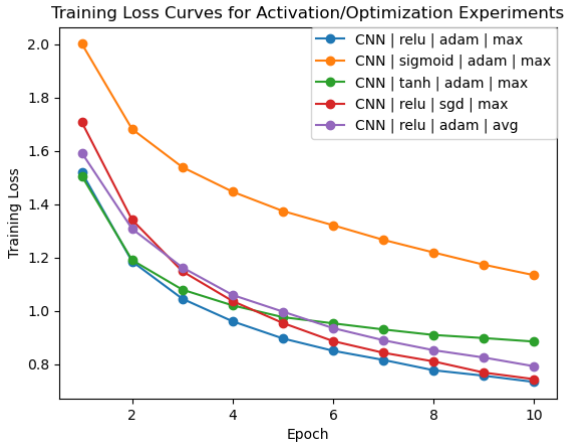

Figure 1: Training Loss Curve for Different Hyperparameter on Shallow CNN

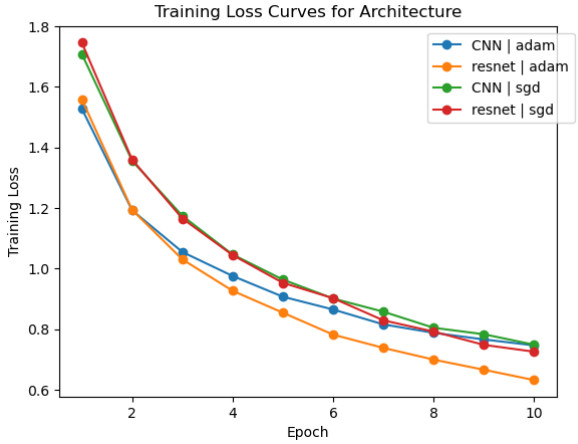

Figure 2: Training Loss Curve for Shallow CNN and ResNet with Optims: Adam and SGD

We can see in Figure 1 that overall best learning uses ReLu with Adam optimizer and max pooling. In Figure 2 we see that ResNet with Adam optimizer surpasses learning in the shallow CNN.

3.2 Training vs Validation Accuracy

We look to see how the networks perform on training and testing sets to observe any overfitting and underfitting.

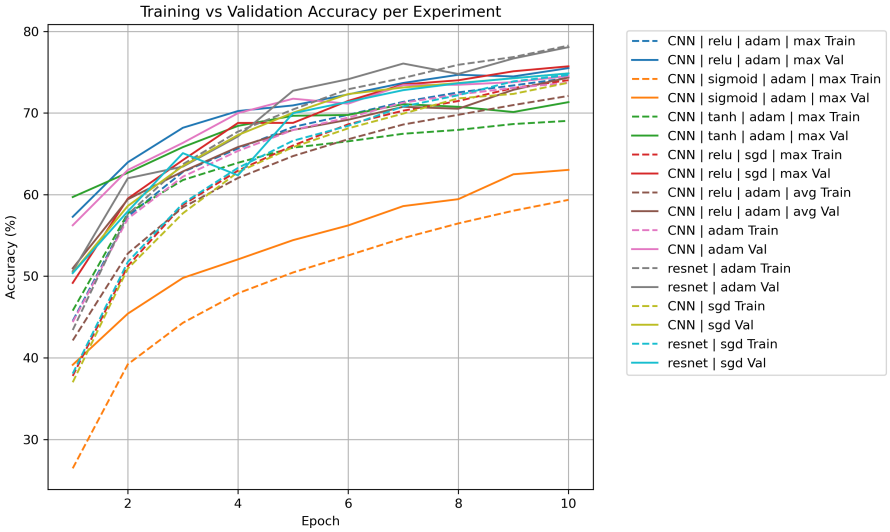

Figure 3: Valid vs. Train Accuracies for all Experiments

3.3 Final Accuracies

Shows the final test accuracies for each experiment

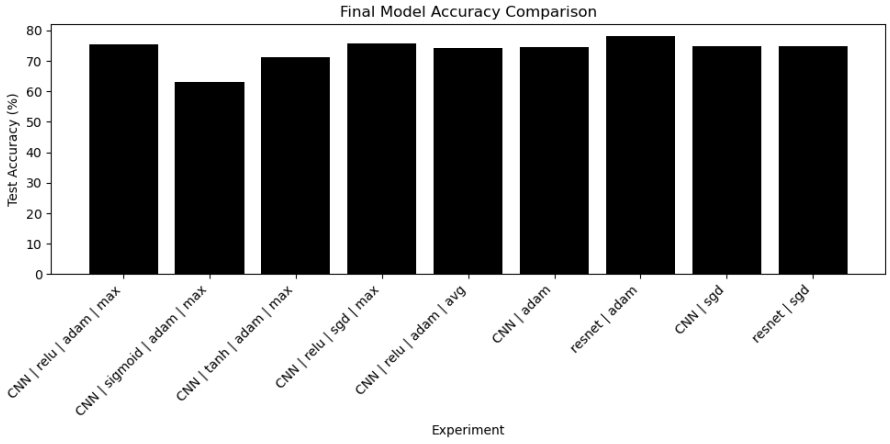

Figure 4: Final Accuracies for Testing

3.4 Confusion Matrices

A set of heatmaps of confusion matrices to see how classification models per experiment perform

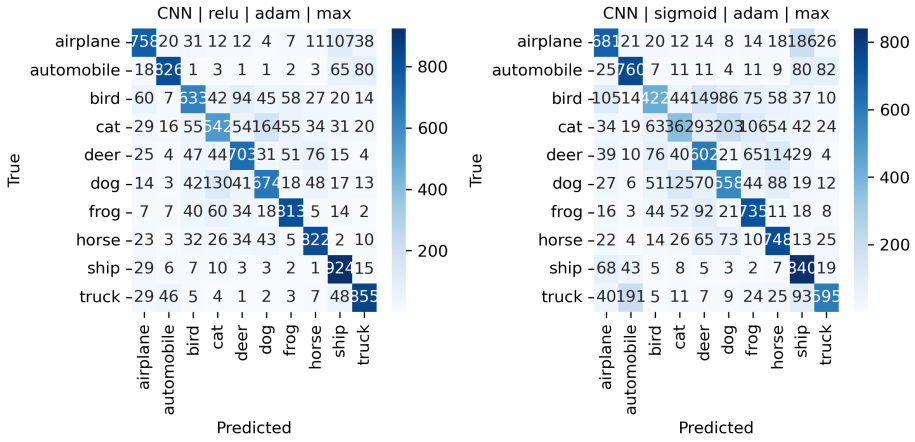

Figure 5: Confusion Matrices for Hyperparameters 1

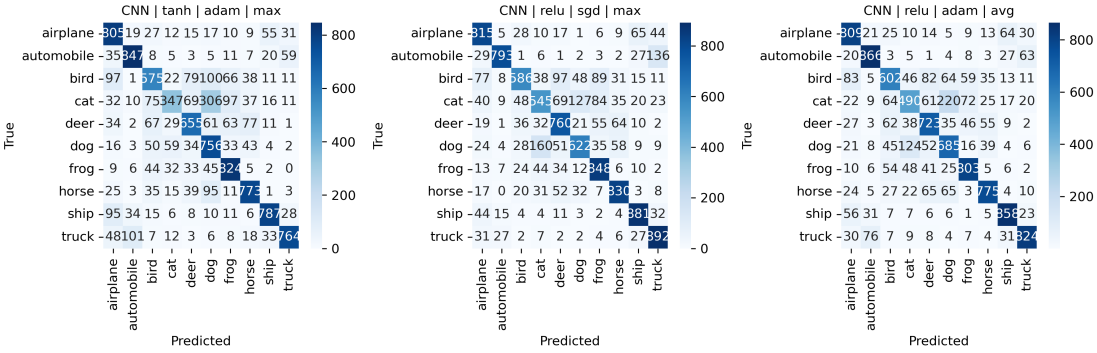

Figure 6: Confusion Matrices for Hyperparameters 2

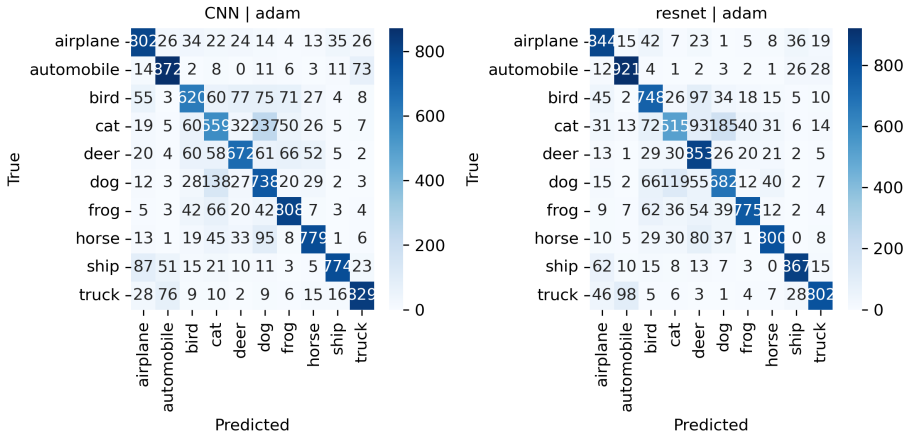

Figure 7: Confusion Matrices for Architectures Adam

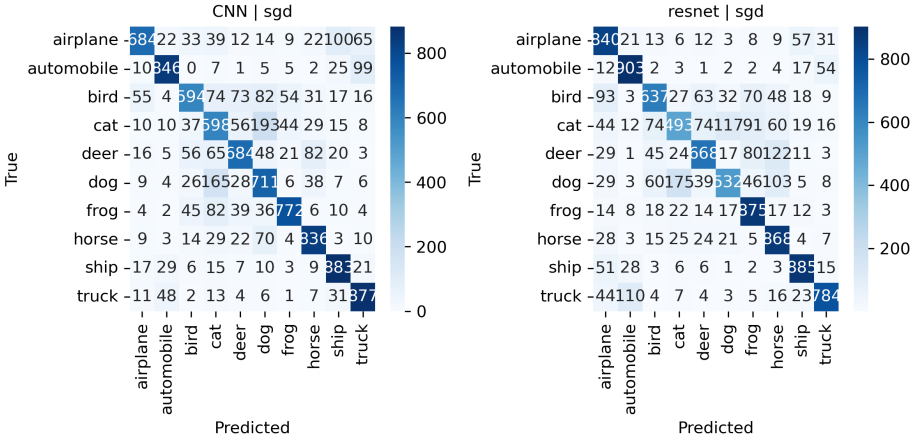

Figure 8: Confusion Matrices for Architectures SGD

4 Discussion and Conclusion

4.0.1 Architecture Differences

The study shows that ResNet is superior to the shallow CNN architecture: after 10 epochs ResNet final accuracy on the Adam optimization is 78.07%, it matches the shallow CNN on SGD at 74.85% accuracy (from Figure 4). The ResNet architecture also shows a smaller training loss curve with the Adam optimizer, suggesting that the ResNet fits the training data faster and better than the simple CNN. ResNet with Adam not only achieved higher training

accuracy but also maintained a slightly higher validation accuracy, suggesting better generalization than the other experiments as well (Figure 2). The diagonal of the heatmap and the off-diagonal cells for the ResNet on both Adam and SGD represent its image classification is superior to the shallow CNN for the most part. These results match what we would expect out of a deeper neural network with more complex parameters and exemplifies the results outlined in [1].

We also see a comparison in the feature maps for each architecture and what the models are actually capturing as an additional research channel. Conv1 for both CNN and ResNet of the feature maps and filter weights appears to focus on the 'big picture' (lower-level) features of each classes image: edges, basic textures, and color coordinates (note the number of filters). The architectures appear to share similar mapping which is to be expected in the first layer, which is responsible for recognizing general features; we can see how each model is attempting to classify and recognize image patterns. The observable differences between the two architectures is likely due to ResNet's deeper learning model which propagates "short-cut connections" that lead to improved feature mapping and learning as the model trains [1]. Ultimately, these differences may suggest that performance differences are due to a models' ability to refine and recombine features in its later layers.

4.0.2 Shallow CNN Hyperparameters

The study suggest that certain hyperparameters outperformed others in the shallow CNN which changed the best overall performance. Hyperparameter tuning showed vast differences in performance depending on specific activation function, optimizer, and pooling: all of which were evaluated on their test accuracy, validation and training accuracy, training loss, and a general overlook at the model's classification scores (heatmap).

The ReLu activation function with 75.5% final accuracy after testing, exceeded both the Sigmoid and Tanh functions (63.03% and 71.33%). Sigmoid performed the worst overall in loss training (see Figure 1) with Tanh performing averagely. Looking at the confusion matrices we continue to see Sigmoid performing the worst in classification with the weakest diagonal. We see that ReLu has the best overall performance out of the three selected activation functions, likely due to its resistance to vanishing gradients and propagation efficiency.

Adam and SGD optimizers appear to have very similar final test accuracies, 75.50% and 75.72% respectively. However, Adam has a steeper training loss curve at each epoch which suggests faster overall convergence of the model as compared to SGD (Adam has an adaptive learning rate which provides a more stable learning structure). We see that SGD has slightly better validation accuracy while Adam has stronger training accuracy: this effect is likely due to Adam overfitting slightly at 10 epochs while SGD performs better generalization which leads to better fitting in validation. Figures 5 and 6 depict a very similar overall classification of the two optimizers which is expected at 10 epochs and the same training dataset.

Finally, we observe the different effects between maximum and average pooling (both of which were run on ReLu and Adam): final test accuracy on max is 75.5% while on avg we have 74.35%. This suggests that pooling strategy doesn't have a great effect on the overall final test accuracy. However, the training loss curve for maximum pooling is steeper than avg with faster overall convergence and less loss. We also see that the validation and training accuracy for max is superior to avg with stronger accuracy curves: this is further emphasized in the confusion matrices as wee see max pooling with a better overall classification diagonal for its model. Maximum value pooling provides stronger feature mapping for a model as it emphasizes the most prominent image features at the strongest neuron with better information updates, thus outperforming average pooling which is ultimately slower and less accurate.

4.1 Limitations

Limitations of this project include the limited number of training epochs: since ResNet is a much deeper architecture, 10 epochs may not fully reflect its overall performance. This limitation also affects the analysis of optimizers, making it difficult to draw strong conclusions about overfitting or underfitting due to the limited training period. The choice of 10 epochs was made to reduce computational resources during model training and testing. Additionally, the study only explores five hyperparameter tuning experiments, which may limit the generalizability of the results.

5 References

[1] He, Kaiming, et al. "Deep Residual Learning for Image Recognition." arXiv.Org, 10 Dec. 2015, arxiv.org/abs/1512.03385.